

Activity 11 - Debug a failing integration with tests

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K4/A4

Specification

`summarise(csv, threshold) -> Result<Summary, String>` where `Summary { lines, total_cents, low_stock }`. Every line counts towards the total; low stock is strictly below the threshold, sorted by SKU; malformed lines are `Err("line N: ...")`.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain assertions, `dbg!`, unit tests with `#[test]`, and reading a failing test to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is `samples/01_dbg_and_assert.rs` from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on assertions, `dbg!`, unit tests with `#[test]`, and reading a failing test with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles testing and debugging with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: `summarise(csv, threshold) -> Result<Summary, String>` where `Summary { lines, total_cents, low_stock }`. Every line counts towards the total; low stock is strictly below the threshold, sorted by SKU; malformed lines are `Err("line N: ...")`. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Debug a failing integration with tests" in `starter/src` so that the supplied tests pass. Give the AI the failing test output and the function only; ask for a diagnosis before any code change. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_dbg_and_assert.rs

Run: `rustc --edition 2021 01_dbg_and_assert.rs && ./01_dbg_and_assert`

```
// Sample: dbg! and assert_eq! as debugging tools.
fn total(values: &[u64]) -> u64 {
    values.iter().sum()
}

fn main() {
    let values = [35_960, 12_495, 0, 4_200];
    let result = dbg!(total(&values)); // prints file:line and value to stderr
    assert_eq!(result, 52_655, "total must include every line");
    let skipped: u64 = values.iter().skip(1).sum();
    println!("correct total {result}, total without the first line {skipped}");
    println!("difference {} = the first line", result - skipped);
}
```

Expected output:

```
correct total 52655, total without the first line 16695
difference 35960 = the first line
```

02_unit_tests.rs

Run: `rustc --edition 2021 --test 02_unit_tests.rs && ./02_unit_tests`

```
// Sample: unit tests in one file. Run with the --test flag (see README).
pub fn low_stock(levels: &[(&str, u32)], threshold: u32) -> Vec<String> {
    let mut skus: Vec<String> = levels
        .iter()
        .filter(|(_, qty)| *qty < threshold)
        .map(|(sku, _)| sku.to_string())
        .collect();
    skus.sort();
    skus
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn strictly_below_threshold() {
        assert_eq!(low_stock(&[("A", 4), ("B", 5)], 5), ["A"]);
    }

    #[test]
    fn sorted_output() {
        assert_eq!(low_stock(&[("B", 0), ("A", 1)], 5), ["A", "B"]);
    }

    #[test]
    fn nothing_is_low_at_zero() {
        assert!(low_stock(&[("A", 0)], 0).is_empty());
    }
}
```

Expected output:

```
test result: ok. 3 passed
```

Learn more

Rust Book: How to Write Tests: <https://doc.rust-lang.org/book/ch11-01-writing-tests.html>

Rust Book: Test Organization: <https://doc.rust-lang.org/book/ch11-03-test-organization.html>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php